

prepare :
[Static Field memory allocation + Static Field initialization with default value even the field is final]

It will allocate the memory for all the Static Fields, here all the Static Fields will get the default values so if we have static int x = 100; then for field x memory will be allocated (4 bytes) and now it will initialize with default value i.e 0, even the field is final.

```
static Test t = new Test();
```

Here, t is a static reference field so for t field (reference variable) memory will be allocated as per JVM implementation i.e for 32 bit JVM (4 bytes of Memory) and for 64 bit (8 bytes of memory) and initialized with null.

Note : All the class level fields are having default value even fields are final.

resolve :

All the symbolic references (#7) will be converted into direct references OR actual reference.

```
javap -verbose FileName.class
```

Note :- By using above command we can read the internal details of .class file.

Initialization :

Here class initialization will takes place. All the static field member will get their actual/original value and we can also use static block for static field member initialization.

Here, In this class initialization phase static block is having SAME PRIORITY so it will executed according to the order.(Top to bottom)

****Can we write a Java Program without main method ?**

```
class WithoutMain //JDK 1.6 OR before
{
    static
    {
        System.out.println("Hello User!!");
        System.exit(0);
    }
}
```

It was possible to write a java program without main method till JDK 1.6V.
From JDK 1.7v onwards, at the time of loading the .class file (Initialization Phase), JVM will verify the presence of main method in the .class file. If main method is not available then it will generate a runtime error that "main method not found in so so class".

How many ways we can load a .class file into JVM memory ?

* We can load the .class file into JVM memory by using following ways :

- a) By using Java Command
- b) By creating the object using constructor
- c) By using Inheritance
- d) By accessing the static member (static field OR static method) of the class
- e) By using Reflection API

a) By using Java Command :

```
public class Test
{
}
```

```
javac Test.java
java Test
```

Here we are making a request to class loader sub system to load Test.class file into JVM memory

b) By creating the object using constructor

Example :
new Test(); //Here First of all Test.class file will be loaded then only object will be created.

c) By using Inheritance

When we load a child class then before child class, Parent class will be loaded first. [Child cannot exist without Parent]

```
class Parent
{
    static
    {
        IO.println("SB of Parent");
    }
}
```

```
class Child extends Parent
```

```
{
    static
    {
        IO.println("SB of Child");
    }
}
```

```
public class ClassLoading
```

```
{
    public static void main(String[] args)
    {
        new Child();
    }
}
```

Output :

SB of Parent
SB of Child

d) By accessing the static member (static field OR static method) of the class

Whenever we try to access the static members using class then first of all JVM will load that class as shown in the program below.

```
class Test
{
    static int x; //x = 0
}
```

```
static
{
    IO.println("SB");
}
```

```
public class ClassLoading
```

```
{
    public static void main(String[] args)
    {
        IO.println("x value is : "+Test.x);
    }
}
```

Output :

SB
x value is : 0

